zkvm TUI Architecture

Interactive Terminal UI for VM Migration

A Go + Bubble Tea terminal application with 5 tabs, guided form-based migration, vSphere discovery, live progress streaming, file browser with fuzzy search, libvirt VM management, and KubeVirt deployment. 10,382 lines of Go powering a zero-learning-curve migration experience.

Go + Bubble Tea | 5 Tabs | 22 Source Files | 10,382 Lines | Proprietary (HyperSDK)

What Is zkvm?

A terminal-based interactive UI that guides operators through VM migration without memorizing CLI flags.

5
Tabs: Migration, vSphere, Logs, Libvirt, Kubernetes

10,382
Lines of Go code across 22 source files

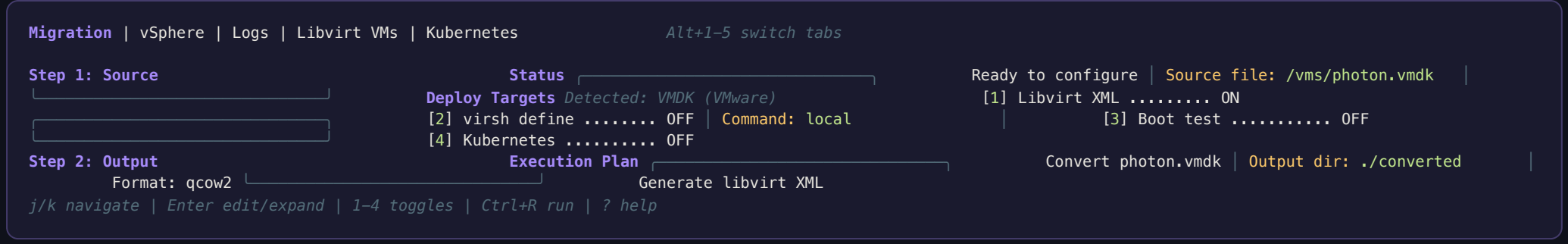
0
CLI flags to memorize guided form does everything

116
Go tests passing across 4 packages

The 5 Tabs

#	Tab	Purpose	Key Features	Source File
1	**Migration**	Main config form — guided step-by-step migration setup	Form fields, deploy toggles (1-4), execution plan, file browser, source auto-detect	`standalone.go` + `form.go`
2	**vSphere**	vCenter connection, VM discovery, batch selection	Env var auto-populate, govc streaming, VM table with CPU/RAM/state, Ctrl+R migrate	`vsphere_tab.go` + `vsphere_discover.go`
3	**Logs**	Full scrollable log viewer	Color-coded log levels, auto-scroll, viewport with PgUp/PgDn, search	`standalone.go` (viewport)
4	**Libvirt VMs**	List and manage local KVM virtual machines	Start/stop/delete VMs, console access (VNC), virsh integration, status refresh	`libvirt.go` + `virsh.go`
5	**Kubernetes**	KubeVirt VM management on any K8s cluster	Namespace selection, VM list, PVC management, multi-context, kubectl/virtctl wrapper	`kubernetes.go` + `kube_client.go` + `kubevirt.go`

What It Looks Like


Migration | vSphere | Logs | Libvirt VMs | Kubernetes
Alt+1–5 switch tabs
Step 1: Source
Status
Deploy Targets Detected: VMDK (VMware)
[2] virsh define ..... OFF | Command: local
[4] Kubernetes ..... OFF
Ready to configure | Source file: /vms/photon.vmdk
[1] Libvirt XML ..... ON
[3] Boot test ..... OFF
Step 2: Output
Format: qcow2
Execution Plan
Generate libvirt XML
Convert photon.vmdk | Output dir: ./converted
j/k navigate | Enter edit/expand | 1–4 toggles | Ctrl+R run | ? help

Architecture — Elm Architecture (TEA)

zkvm uses Bubble Tea's Elm Architecture: Model holds state, Update handles events, View renders.

```
// The Elm Architecture (TEA) loop

Init() → initial Model + initial Cmd
     ↓
┌──────────────────────────────────────────┐
│  Update(msg) → new Model + new Cmd       │ ← tea.KeyMsg, tea.WindowSizeMsg,
│       ↓                                  │   custom Msgs (VMListMsg, LogMsg, ...)
│  View() → string (rendered terminal)     │ → lipgloss-styled output to terminal
│       ↓                                  │
│  Terminal displays rendered output       │
│       ↓                                  │
│  User presses key → tea.KeyMsg           │
└──────────────────────────────────────────┘
     ↑                    ↓
     └────────────────────┘  // Loop forever

// Key files in the loop:
// standalone.go:  Model struct, Update(), View(), handleKey()
// form.go:        FormData, categories, fields, input box rendering
// runner.go:      h2kvmctl subprocess, progress streaming
// vsphere_tab.go: vSphere connection and discovery UI
```

Source File Map (22 files, 10,382 lines)

File	Lines	Responsibility	Key Types / Functions
standalone.go	1,781	Top-level Model, Update/View loop, tab routing, key handling, layout	`Model`, `Update()`, `View()`, `handleKey()`, `handleMigrationKey()`

form.go	1,155	Form data model, categories, fields, input box rendering, BuildYAML	`FormData`, `Category`, `Field`, `BuildYAML()`, `detectSourceFormat()`
kubernetes.go	1,193	Kubernetes tab: KubeVirt VM management, namespace, PVC	`KubernetesTab`, `handleKubernetesKey()`
virsh.go	839	virsh command wrapper for libvirt operations	`virshList()`, `virshStart()`, `virshDefine()`
libvirt.go	604	Libvirt VMs tab: list, start, stop, delete, console	`LibvirtTab`, `handleLibvirtKey()`
kube_client.go	494	kubectl/virtctl wrapper for K8s operations	`KubeClient`, `getVMs()`, `getPVCs()`
filebrowser.go	408	Modal file picker with fuzzy search	`FileBrowser`, `fuzzyMatch()`, extension filtering
runner.go	367	h2kvmctl subprocess, progress streaming, auto-sudo	`runMigration()`, `scanLinesOrCR()`, progress parsing
vsphere_discover.go	355	VSphereClient, govc wrapper, VM discovery	`VSphereClient`, `discoverVMs()`, streaming output
vsphere_tab.go	308	vSphere tab UI, env var reading, connection fields	`*VsphereTab` (pointer!), env auto-populate
diagnosis.go	231	Failure analysis and troubleshooting hints	`diagnoseFailure()`, error pattern matching
help.go	206	Help overlay with keyboard shortcuts	`helpView()`, key binding reference
quick_migrate.go	200	Auto-detect source format and run migration	`quickMigrate()`, format detection
console.go	190	VNC/noVNC console access for VMs	`openConsole()`, browser launch
preflight.go	175	Pre-migration system checks	`preflightCheck()`, dependency validation
profiles.go	137	Save/load YAML migration profiles	`saveProfile()`, `loadProfile()`
notifications.go	96	Desktop notifications on completion	`notify()`, libnotify integration
kubevirt.go	129	KubeVirt-specific VM operations	`startVM()`, `stopVM()`, `deleteVM()`

Form System — Guided Migration

The Migration tab uses a category/field form model with input boxes, auto-detection, and YAML generation.

Form Data Model

```
type FormData struct {
    Categories  []Category     // All categories (some hidden via ShowWhen)
    FocusCat    int            // Index into visible categories (clamped)
    FocusField  int            // Index into focused category's fields (reset on collapse)
    InCategory  bool           // true = navigating fields, false = on category header
}

type Category struct {
    Name      string           // "Step 1: Source", "Step 2: Output", etc.
    Expanded  bool             // Whether fields are visible
    Fields    []Field          // Form fields in this category
    ShowWhen  func() bool     // Conditional visibility (e.g., vSphere only when cmd=vsphere)
}

type Field struct {
    Label        string         // Display label
    Key          string         // YAML key for BuildYAML()
    Value        string         // Current value
    Editing      bool           // Currently in text edit mode
    FieldType    FieldType      // Text, Select, Toggle, PathComplete
    Options      []string       // For Select type (e.g., ["local","vsphere","hyperv"])
    PathComplete bool           // Opens file browser on Enter
    Extensions   []string       // File browser filter (e.g., [".vmdk",".ova"])
}
```

Form Categories (Guided Steps)

Category	Default	Fields	Behavior
Step 1: Source	Expanded	Source file path, Command select (local/vsphere/hyperv)	Source auto-detects format (.vmdk/.ova/.vhd/.raw). Sets cmd select to match. File browser with extension filter.

Remote Access	Collapsed	SSH host, SSH user, SSH key path	For remote VM disk access. Collapsed by default — most users use local.
Step 2: Output	Expanded	Output directory, Output format select	Format: qcow2 (default), raw. PathComplete for directory picker.
Step 3: Fixes	Collapsed	Rebuild initramfs toggle, fstab fix toggle	Guest OS fixes. Collapsed because defaults are usually correct.
Step 4: Deploy	Expanded	VM name, memory, vCPUs, machine type, UEFI toggle	Libvirt deployment config. All fields visible for easy customization.
Advanced	Collapsed	Compress, flatten, checksum, dry_run, verbose	Power-user options hidden by default.
Windows	Collapsed	Windows toggle, guest OS, VirtIO ISO path, win_stage	ShowWhen: only visible when Windows migration is detected/enabled.
vSphere	Collapsed	vCenter URL, vs_action, vs_vm name	ShowWhen: only visible when cmd=vsphere. Auto-populated from GOVC_* env vars.
Kubernetes	Collapsed	K8s namespace, deploy_k8s toggle	KubeVirt deployment config.

Navigation & Key Handling

Three-level navigation: tabs, categories, and fields — all keyboard-driven.

```
Update() → tea.KeyMsg → handleKey()
  ├─ Help overlay active? → intercept all keys (Esc to close)
  ├─ Global keys:
  │    ├─ Ctrl+C / Ctrl+Q → quit
  │    ├─ ? → toggle help overlay
  │    ├─ Alt+1-5 → switch tab directly
  │    ├─ Esc+Tab → next tab
  │    └─ Shift+Tab → previous tab
  └─ Route by activeTab:
       ├─ Tab 1 (Migration) → handleMigrationKey()
       │    ├─ FileBrowser active? → route to browser (Enter/Esc/type)
       │    ├─ Field Editing = true?
       │    │    ├─ j / k → EXIT edit mode + navigate (not insert text!)
       │    │    ├─ Tab → move to next field
       │    │    ├─ Esc / Enter → stop editing
       │    │    └─ default → insert character into field value
       │    └─ Form navigation (handleFormKey()):
       │         ├─ 1-4 → toggle deploy targets (Libvirt XML, virsh, boot, K8s)
       │         ├─ j / k → move down/up (categories or fields)
       │         ├─ Enter → expand/collapse category OR start editing OR open file browser
       │         ├─ h / l → cycle select options left/right
       │         ├─ Esc → exit field → collapse category
       │         └─ Ctrl+R → run migration
       ├─ Tab 2 (vSphere) → handleVSphereKey()
       ├─ Tab 3 (Logs) → handleLogsKey() (viewport scroll)
       ├─ Tab 4 (Libvirt) → handleLibvirtKey()
       └─ Tab 5 (Kubernetes) → handleKubernetesKey()
```

Navigation State Machine

Category Level (InCategory=false)

- **j/k** — move between category headers
- **Enter** — expand category, enter field level
- **Esc** — collapse current category
- FocusCat is clamped to visible categories
- Hidden categories (ShowWhen=false) are skipped

Field Level (InCategory=true)

- **j/k** — move between fields within category
- **Enter** — start editing text field, or open file browser for PathComplete
- **h/l** — cycle Select field options
- **Esc** — exit to category level
- FocusField is reset to 0 when category collapses

Edit Mode (Editing=true)

- **Type** — inserts characters into field value
- **Backspace** — deletes last character
- **j/k** — EXIT edit mode and navigate (critical fix!)
- **Tab** — move to next field
- **Esc/Enter** — stop editing, preserve value

Deploy Target Toggles (Right Panel)

Key	Toggle	YAML Key	Effect
1	Libvirt XML	`emit_domain_xml: true`	Generate libvirt domain XML file
2	virsh define	`virsh_define: true`	Auto-run virsh define after conversion
3	Boot test	`boot_test: true`	Start VM and verify it boots
4	Kubernetes	`deploy_k8s: true`	Generate KubeVirt manifest and apply

vSphere Integration

Discover, select, and migrate VMs from vCenter — all from the TUI.

Env Vars (GOVC_*) → Connect to vCenter → Discover VMs → Select Batch → Ctrl+R Migrate

VsphereTab (vsphere_tab.go)

- **Pointer type (*VsphereTab)** — survives Bubble Tea's value copies
- Auto-populates from env vars: GOVC_URL, GOVC_USERNAME, GOVC_PASSWORD, GOVC_DATACENTER
- Fields show `[env]` tag when populated from environment
- Connection status indicator (connected/disconnected)
- VM table with columns: Name, CPU, RAM, Power State, Guest OS
- Multi-select: space to toggle, Ctrl+A select all

VSphereClient (vsphere_discover.go)

- Wraps `govc` CLI (cached path lookup)
- `govc ls /datacenter/vm/` for VM inventory
- `govc vm.info -json` for detailed metadata (batch of 10)
- 30-second timeout per discovery operation
- Streaming output: VMs appear as they're discovered
- Error handling: connection failures, auth errors, timeout

Migration Flow (Ctrl+R)

- syncVSphereToForm() copies selected VMs to Migration tab
- Auto-sets `cmd: vsphere` and `vs_action: export_vm`
- Auto-enables `emit_domain_xml` and `regen_initramfs`
- Generates YAML config via BuildYAML()
- Launches h2kvmctl subprocess via runner.go
- Progress streaming in Logs tab + progress bar in right panel

Progress Streaming (runner.go)

- `scanLinesOrCR()` splits on both \n and \r for govc progress
- Throttled: govc download lines shown once per 3 seconds
- Dedup + rate limit prevents log flooding
- Structured `[PROGRESS]` lines: pct|current|total|rate|eta
- Progress bar: `[PROGRESS] 45% 2.3GB/5.1GB 120MB/s ETA 23s`
- 5-sample moving average for smoothed speed display

Auto-Sudo for vSphere Exports

```
// runner.go: Auto-prepend sudo when not running as root
func runMigration(config string) *exec.Cmd {
    args := []string{"--config", configPath}
    if os.Getuid() != 0 {
        // Prepend sudo -E to preserve GOVC_* env vars
        args = append([]string{"-E", "h2kvmctl"}, args...)
```

```
        return exec.Command("sudo", args...)
    }
    return exec.Command("h2kvmctl", args...)
}
```

File Browser & Special Features

Built-in fzf-style file picker, source auto-detection, and YAML generation.

File Browser (filebrowser.go, 408 lines)

- Opens on **Enter** for PathComplete fields
- **Fuzzy search:** `/` to activate, type to filter instantly
- **Match highlighting:** matched characters shown with underline
- **Result count:** "query (N results)" shown in search bar
- **Ctrl+H** toggles hidden files (.dotfiles)
- **~** jumps to home directory
- **/** jumps to filesystem root
- **Backspace** navigates up one directory
- **Extension filtering** per field (.vmdk, .ova, .ovf, .vhd, .raw, .iso)
- Modal overlay at bottom of screen — doesn't replace the form

Source Auto-Detection (form.go)

- `detectSourceFormat()` maps file extension to format
- `.vmdk` → "VMDK (VMware)" → YAML key: `vmdk:`
- `.ova` → "OVA (VMware)" → YAML key: `ova:`
- `.ovf` → "OVF (VMware)" → YAML key: `ovf:`
- `.vhd/.vhdx` → "VHD (Hyper-V)" → YAML key: `vhd:`
- `.raw/.img` → "RAW" → YAML key: `raw:`
- Hint shown below field: "Detected: VMDK (VMware)"
- Also auto-sets `cmd` select to match source type

BuildYAML() — Config Generation

- Reads all form fields and generates valid h2kvmctl YAML
- Source field uses detected format key (vmdk/ova/vhd/raw)
- Toggle fields emit `true` / `false`
- Empty fields omitted from YAML
- vSphere category only emitted when `cmd: vsphere`
- Deploy toggles (1-4) add corresponding YAML keys
- Written to temp file, passed to h2kvmctl --config

Additional Features

- **Profiles (profiles.go):** Save/load YAML configs for reuse
- **Quick Migrate (quick_migrate.go):** Auto-detect source and run with defaults
- **Pre-flight (preflight.go):** Check dependencies before migration
- **Diagnosis (diagnosis.go):** Analyze failures with pattern matching
- **Console (console.go):** Open VNC/noVNC console in browser
- **Notifications (notifications.go):** Desktop notify on completion
- **Help (help.go):** `?` shows full keyboard shortcut reference

Rendering — Input Box Layout

```
// Each field renders as 4 lines (critical for FocusRow scroll calculation):
// Line 1: Label
// Line 2: ┌──────────────────────────┐  (top border)
```

```
// Line 3: │ field value              │  (content)
// Line 4: └──────────────────────────┘  (bottom border)

// FocusRow() counts: (field_index * 4) + category headers above
// This ensures viewport scrolls to keep the focused field visible

// Right panel layout (status-only, no logs):
// ┌──────────────────────┐
// │ Status               │  Ready / Running / Complete
// │ Deploy Targets       │  [1] XML [2] virsh [3] boot [4] K8s
// │ Execution Plan       │  Dynamic based on current settings
// │ Progress Bar         │  ████▒▒▒▒ 45% 120MB/s ETA 23s
// │ Shortcuts            │  Key bindings for current context
// └──────────────────────┘
```

Libvirt & Kubernetes Tabs

Post-migration management: manage KVM VMs and KubeVirt deployments from the same TUI.

Tab 4 Libvirt VMs (libvirt.go + virsh.go)

- **VM List:** Table with name, state (running/shut off), CPU, RAM, autostart
- **virsh integration:** Wraps virsh commands — list, start, shutdown, destroy, undefine, dominfo
- **Start/Stop:** `s` to start, `S` to shutdown, `D` to destroy (force stop)
- **Delete:** `x` to undefine + remove disk (with confirmation)
- **Console:** `c` opens VNC viewer or noVNC in browser
- **Refresh:** `r` refreshes VM list from libvirt
- **Detail view:** Enter shows full VM info (CPU, RAM, disks, NICs, XML path)
- Auto-refresh after migration completes in Migration tab

Tab 5 Kubernetes (kubernetes.go + kube_client.go + kubevirt.go)

- **KubeVirt VMs:** List VirtualMachines with status, CPU, RAM, node
- **PVC management:** List PersistentVolumeClaims with size, status, storage class
- **Namespace selector:** `n` to switch namespace
- **Multi-context:** `k` to switch kubeconfig context (multiple clusters)
- **VM operations:** Start, stop, restart, delete KubeVirt VMs
- **Console:** `c` opens virtctl console or VNC
- **kubectl wrapper:** Runs kubectl/virtctl commands, parses JSON output
- **Auto-detection:** Checks if KubeVirt is installed on the cluster

Logs Tab

Tab 3 Full Log Viewer

- Scrollable viewport with Bubble Tea viewport component
- Color-coded log levels: ERROR (red), WARN (yellow), INFO (blue), DEBUG (gray)
- Auto-scroll to bottom during active migration
- PgUp/PgDn for page scrolling, Home/End for top/bottom
- All h2kvmctl output streamed here in real-time
- [PROGRESS] lines filtered out (shown as progress bar in right panel instead)
- Logs persist across tab switches — switch to Logs tab anytime to check history

Help Overlay

- Press `?` anywhere to show/hide help overlay
- Context-sensitive: shows keys relevant to current tab
- Global shortcuts always shown (tab switching, quit, help)
- Migration tab: form navigation, deploy toggles, run
- vSphere tab: connect, discover, select, migrate
- Libvirt/K8s tabs: VM operations, console, refresh
- Press `Esc` or `?` again to dismiss

Message Types (Bubble Tea Commands)

Message Type	Source	Triggers
`tea.KeyMsg`	User keyboard input	handleKey() routing to active tab handler
`tea.WindowSizeMsg`	Terminal resize	Recalculate layout, viewport, form width
`VMListMsg`	virsh list / kubectl get	Update Libvirt or Kubernetes tab VM table
`LogMsg`	runner.go stdout/stderr	Append to log viewport, parse [PROGRESS] lines
`VSphereDiscoverMsg`	govc vm.info	Add discovered VMs to vSphere tab table
`MigrationDoneMsg`	h2kvmctl exit	Update status, refresh Libvirt/K8s tabs, notify
`FileBrowserMsg`	File browser selection	Set field value to selected file path, close browser

Why zkvm Matters

A TUI that makes VM migration accessible to any operator — not just CLI experts.

Zero Learning Curve

Operators don't memorize `h2kvmctl --vmdk /path --out-format qcow2 --emit-domain-xml --vm-name foo --memory 4096 --vcpus 2`. They fill in a form, toggle deploy targets, press Ctrl+R. The TUI builds the YAML and runs the command.

Complete Workflow in One Screen

Discover VMs on vSphere (tab 2), configure migration (tab 1), watch progress (tab 3), verify KVM VMs (tab 4), check KubeVirt (tab 5). No context switching between 5 different CLI tools.

No Other Migration Tool Has This

virt-v2v: CLI only. MTV: web UI (requires OpenShift console). AWS MGN: web console. Carbonite: Windows GUI. **zkvm is the only TUI for VM migration.** Works over SSH, in tmux, in any terminal.

Production-Quality Code

10,382 lines of Go. 116 passing tests. Elm Architecture via Bubble Tea. Lipgloss styling. Clean separation: form model, tab handlers, subprocess runner, file browser. All critical navigation bugs fixed and tested.

22

Go source files in standalone/ package

5

Tabs: Migration, vSphere, Logs, Libvirt, Kubernetes

9

Form categories with conditional visibility

4

Deploy toggles: XML, virsh, boot, Kubernetes

zkvm: The Migration TUI

Guided form-based configuration. vSphere discovery with batch selection.
File browser with fuzzy search. Live progress streaming. Boot verification.

Libvirt and KubeVirt management. All in a terminal. All keyboard-driven.

The easiest way to migrate VMs — no CLI flags, no web browser, no learning curve.